other hand, you don’t want to fill your logs with data that makes it hard to find the important information when you need it. Even worse, what is sufficient in development might be far too much once you’re running in production.

In section 26.4 I’ll explain how you can filter log messages from various sections of your app, such as the ASP.NET Core infrastructure libraries, so that your logging providers write only the important messages. This lets you keep that balance between extensive logging in development and writing only important logs in production.

In the final section of this chapter I’ll touch on some of the benefits of structured logging, an approach to logging that you can use with some providers for the ASP.NET Core logging framework. *Structured logging* involves attaching data to log messages as key-value pairs to make it easier to search and query logs. You might attach a unique customer ID to every log message generated by your app, for example. Finding all the log messages associated with a user is much simpler with this approach, compared with recording the customer ID in an inconsistent manner as part of the log message.

We’ll start this chapter by digging into what logging involves and why your future self will thank you for using logging effectively in your application. Then we’ll look at the pieces of the ASP.NET Core logging framework you’ll use directly in your apps and how they fit together.

26.1 Using logging effectively in a production app